MODULE IV

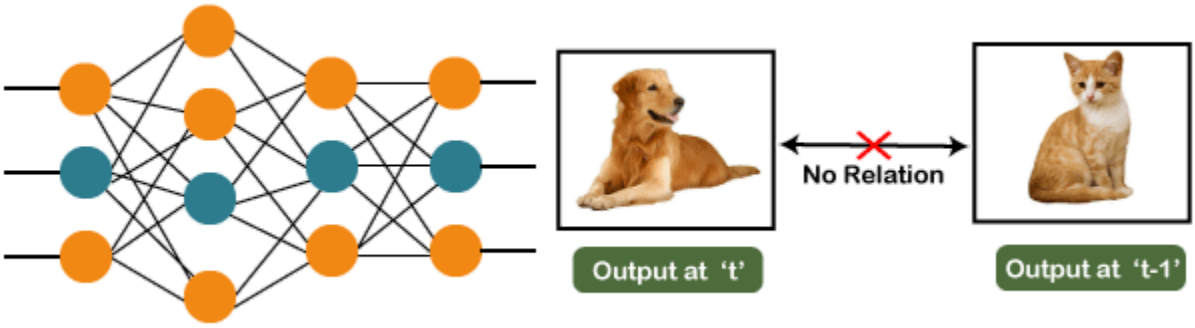
Recurrent Neural Networks

Why not Feedforward Networks?

Feedforward networks are used to classify images. If we feed an image of a cat, it will identify that image and provide a relevant label to that particular image. Similarly, if you feed an image of a dog, it will provide a relevant label to that image a particular image as well.

Consider the following diagram:

And if you notice the new output that we have got is classifying, a dog has no relation to the previous output that is of a cat, or you can say that the output at the time '**t**' is independent of output at a time '**t-1**'. It can be clearly seen that there is no relation between the new output and the previous output. So, we can say that in feedforward networks, the outputs are independent of each other.



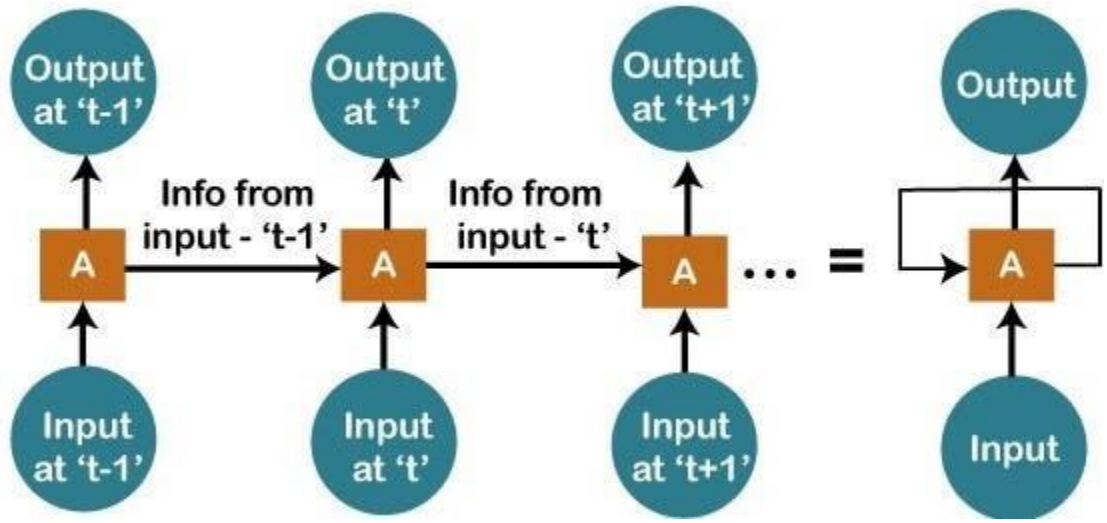
There are a few scenarios where we will actually need the previous output to get the new output.

In the feedforward network, the new output is independent of the previous outputs, i.e., output at '**t+1**' has no relation with the output at '**t-2**', '**t-1**', and '**t**.' Therefore, it can be concluded that we cannot use feedforward networks for predicting the next word in the sentence.

Similarly, many other examples can also be taken where we need the previous output or some information from the previous output, so as to infer the new output.

How to overcome this challenge?

Consider the following diagram:

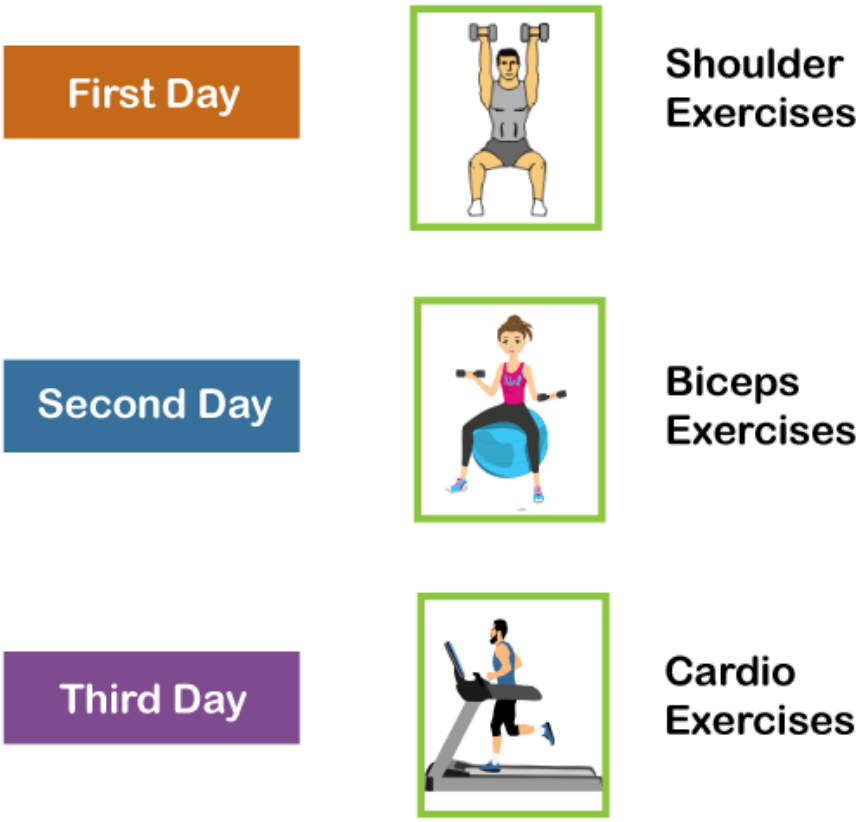


We have input at '**t-1**', which we will feed to the network, and then we will get the output at '**t-1**'. Then at the next timestamp that is at a time '**t**', we have an input at a time '**t**', which will be again given to the network along with the information from the previous timestamp. Similarly, at the output for '**t+1**', we have two inputs; one is the new input that we give, and the other is the information coming from the previous timestamps, i.e., '**t**' in order to get the output at a time '**t+1**'.

What are Recurrent Neural Networks?

"Recurrent Networks are one such kind of artificial neural network that are mainly intended to identify patterns in data sequences, such as text, genomes, handwriting, the spoken word, numerical times series data emanating from sensors, stock markets, and government agencies".

In order to understand the concept of Recurrent Neural Networks, let's consider the following analogy.

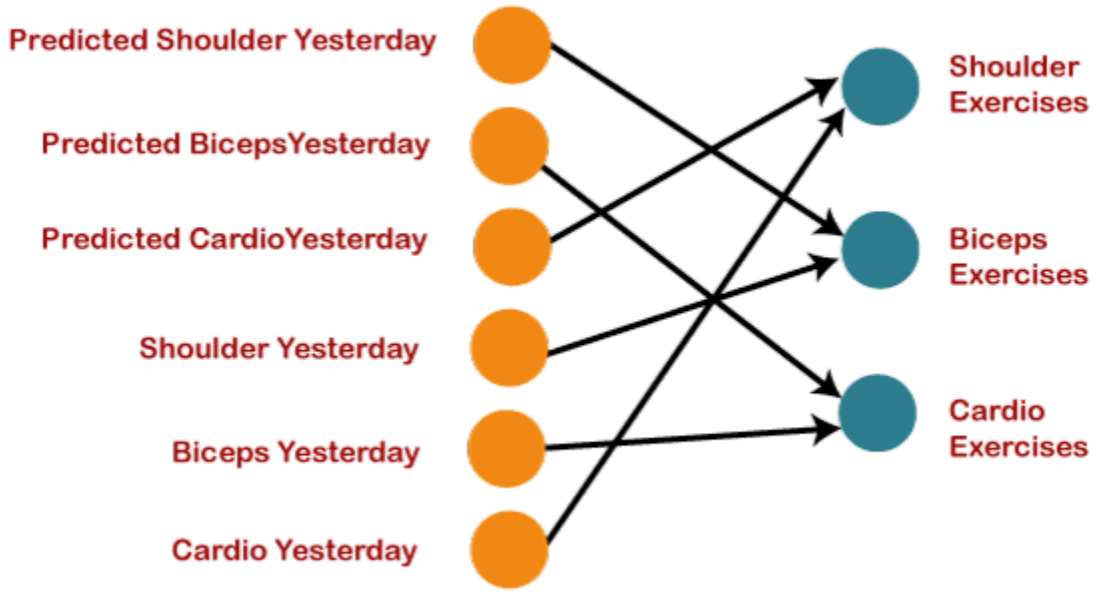


Suppose that your gym trainer has made a schedule for you. The exercises are repeated every third day. The above image includes the order of your exercises; on your very first day, you will be doing shoulders, the second day you will be doing biceps, the third day you will be doing cardio, and all these exercises are repeated in proper order.

Therefore, if shoulders were done yesterday, then definitely today will be biceps day. Similarly, if biceps were done yesterday, then today will be the cardio day, and if yesterday was the cardio day, then today, we will need to undergo shoulder.

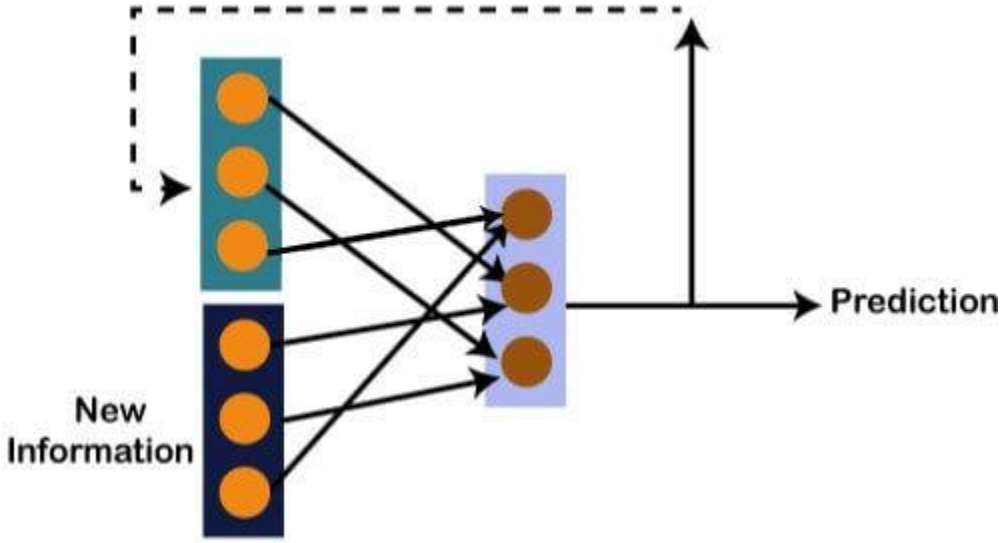
Now there can be one such scenario, where you are unable to go to the gym for one day due to some personal reasons, then it that case, we will

go one timestamp back and will feed in what exercise happened day before yesterday as shown below.

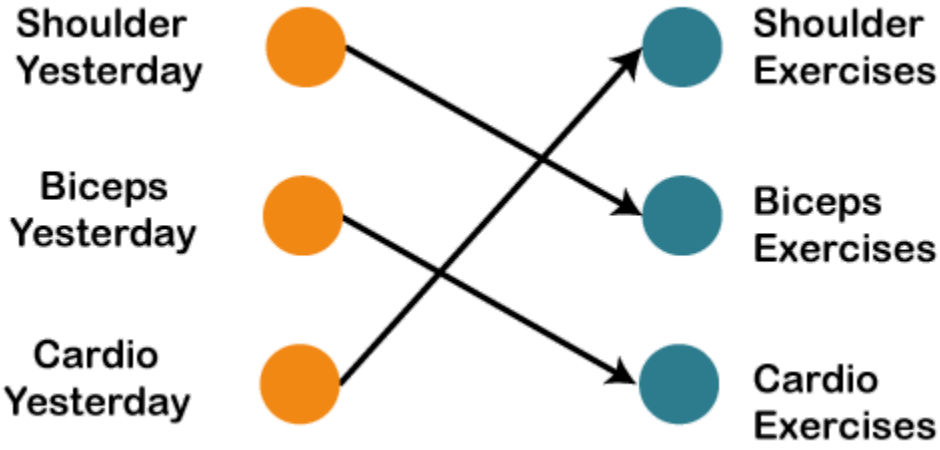


So, if the exercise that happened the day before yesterday was the shoulder, then yesterday there were biceps exercises. Similarly, if biceps happened the day before yesterday, then yesterday would have been cardio exercises, and if cardio would have happened the day before yesterday, then yesterday would have been shoulder exercises. And this prediction for the exercises that happened yesterday will be fed back to our network so that these predictions can be used as inputs in order to predict what exercise will happen today. Similarly, if you have missed your gym say for two days, three days or even one week, you will actually need to roll back, which means that you will need to go to the last day when you went to the gym, you need to figure out what exercises you did on that day and then only you will be getting the relevant output as to what exercises will happen today.

Next, we will convert all these things into a vector, which is nothing but a list of numbers.

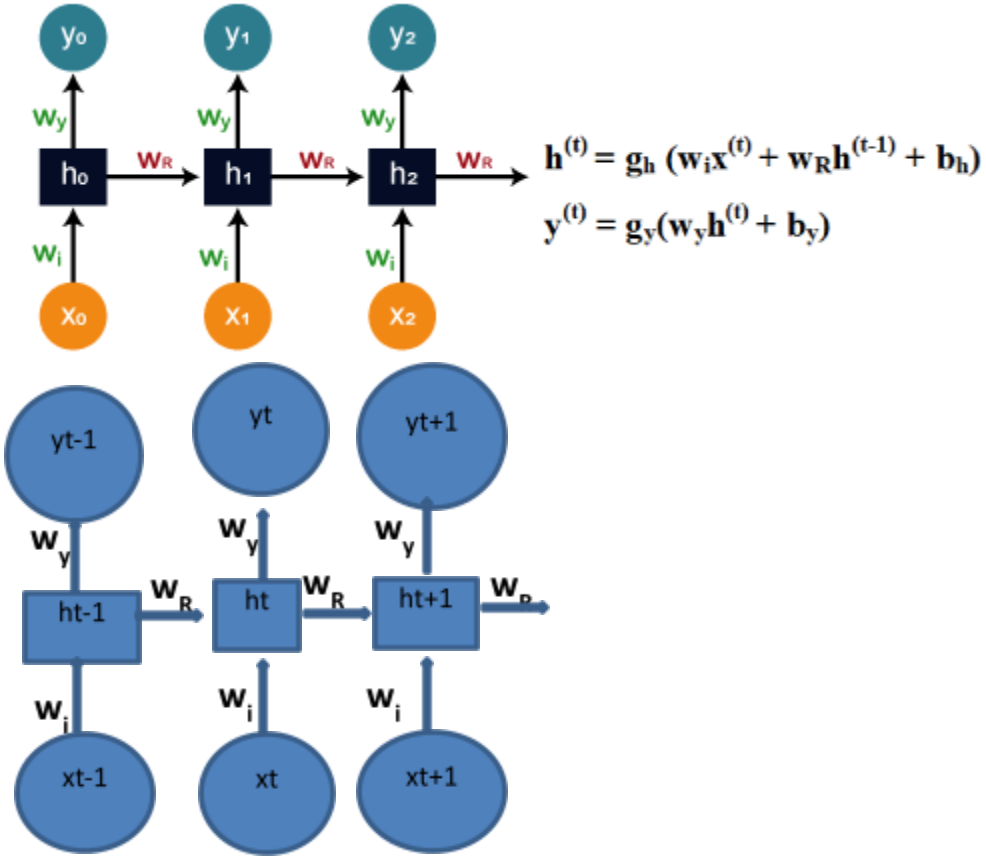


So, there is new information along with the information which we got from the prediction at the previous timestamp because we need all of these in order to get the prediction at a time '**t**'. Imagine that you did shoulder exercises yesterday, then, in that case, the prediction will be biceps exercise because if the shoulder was done yesterday, then today it will definitely be biceps and output will be **0, 1**, and **0**, which is actually the work of our vectors.



Let's understand the math behind the Recurrent Neural Network

by simply having a look at the image given below.



Assume that '**w**' is the weight matrix, and '**b**' is the bias. Consider at time **t=0**, our input is '$\mathbf{x_o}$', and we need to figure out what exactly is the '$\mathbf{h_o}$'. We will substitute **t=0** in the equation, as shown in the image, so as to procure the function $\mathbf{h^t}$ value.

Training a Recurrent Neural Network

A recurrent neural network uses a backpropagation algorithm for training, but backpropagation happens for every timestamp, which is why it is commonly called as backpropagation through time. With backpropagations, there are certain issues, namely vanishing and exploding gradients.

STUDY vanishing and exploding gradients in previous classes.

Truncated backpropagation through time

Backpropagation Through Time, or BPTT, is the training algorithm used to update weights in recurrent neural networks.

Backpropagation Training Algorithm

1. Present a training input pattern and propagate it through the network to get an output.
2. Compare the predicted outputs to the expected outputs and calculate the error.
3. Calculate the derivatives of the error with respect to the network weights.
4. Adjust the weights to minimize the error.
5. Repeat.

Backpropagation Through Time

Backpropagation Through Time, or BPTT, is the application of the Backpropagation training algorithm to recurrent neural network applied to sequence data like a time series.

The algorithm as follows:

1. Present a sequence of timesteps of input and output pairs to the network.
2. Unroll the network then calculate and accumulate errors across each timestep.
3. Roll-up the network and update weights.
4. Repeat.

BPTT can be computationally expensive as the number of timesteps increases. If input sequences are comprised of thousands of timesteps, then this will be the number of derivatives required for a single update weight update. This can cause weights to vanish or explode and make slow learning.

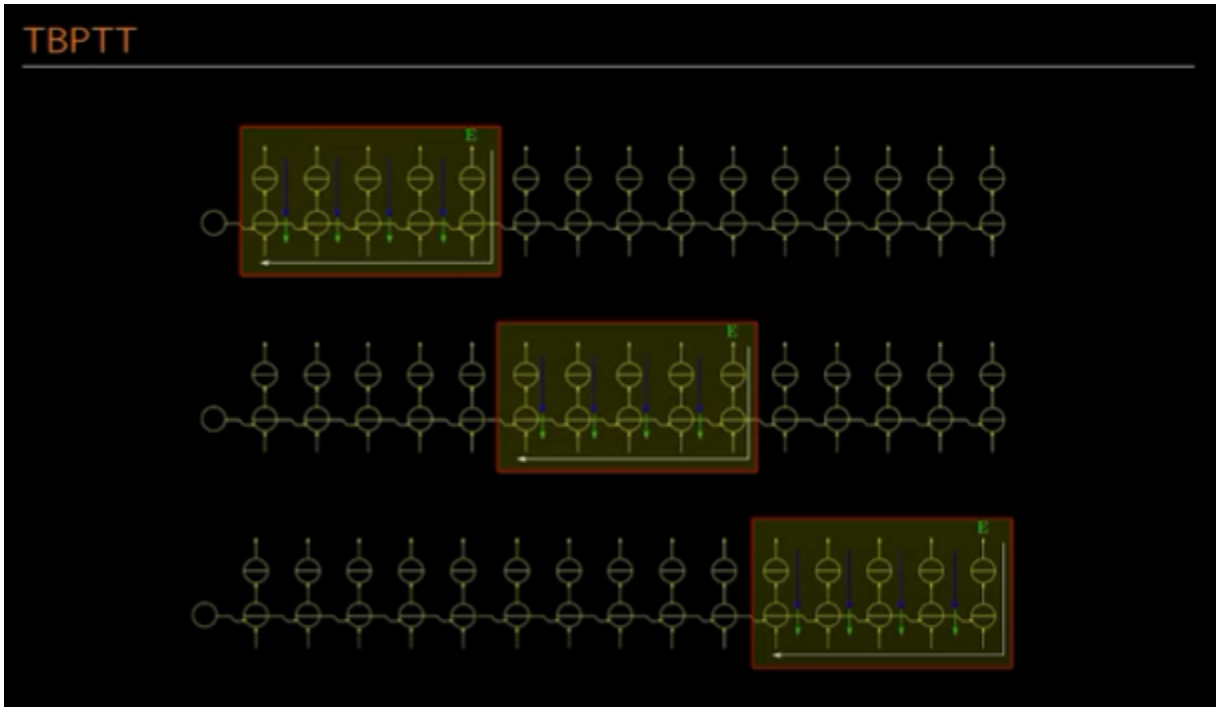
Truncated Backpropagation Through Time

Truncated Backpropagation Through Time, or TBPTT, is a modified version of the BPTT training algorithm for recurrent neural networks where the sequence is processed one timestep at a time and periodically the BPTT update is performed back for a fixed number of timesteps.

For applications with the long sequences the input is translated into manageable fixed sized segments and this approach is called truncated backpropagation through time.

Example

Consider a sequence of 5000 samples. We could split this into 50 sequences of hundred samples each and BPTT is computed for each sequence.

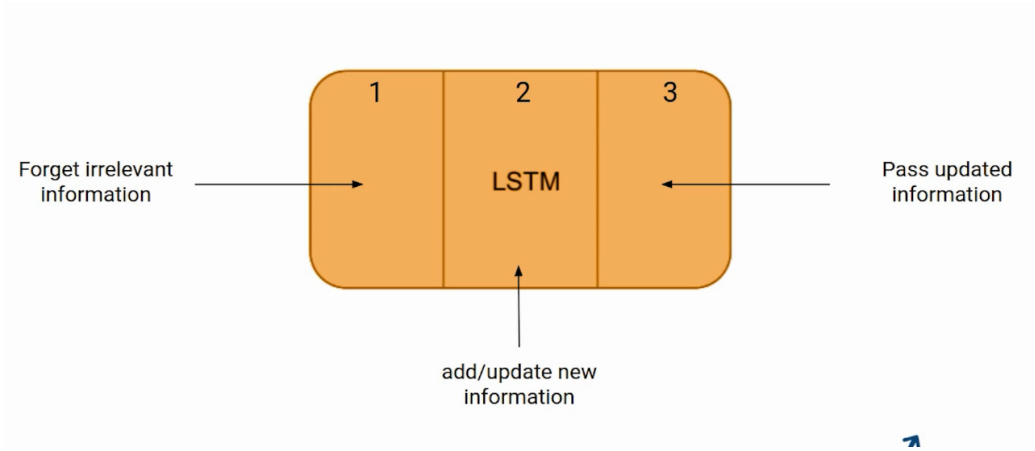


If we have a long sequence, instead of doing back propogation or forward pass till the end of the sequence, do the error calculations and back propogation on chunk. Once chuck1 is computed, feed the value to the next chunk as input. Hence the hidden values of the previous chunks are fed as the output to the next chunk. Move on to the next chunk and so on. This is truncated backpropogation through time.

Long Short-Term Memory (LSTM) cells

Long Short Term Memory Network is an advanced RNN, a sequential network that allows information to persist. It is capable of handling the vanishing gradient problem faced by RNN.

The LSTM consists of three parts, as shown in the image below and each part performs an individual function.

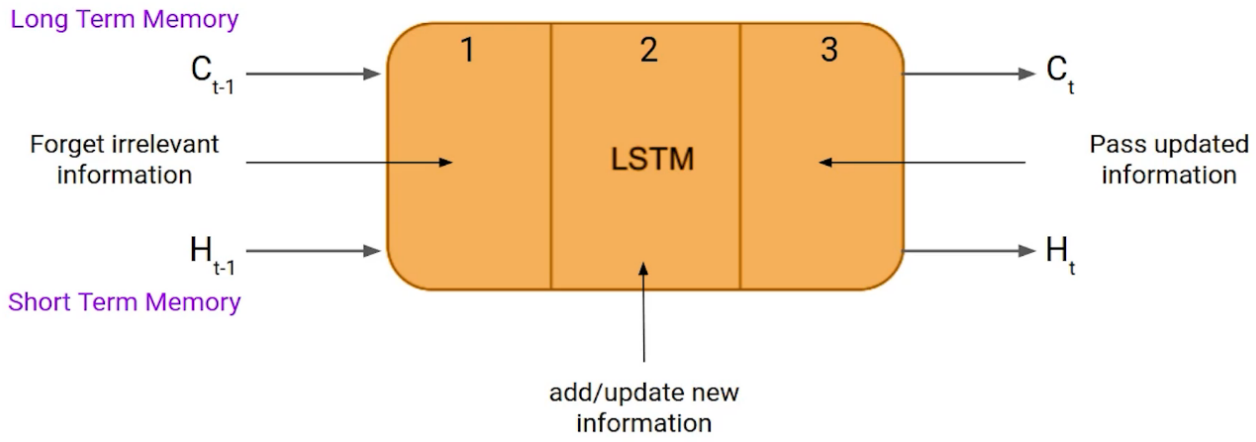


The first part chooses whether the information coming from the previous timestamp is to be remembered or is irrelevant and can be forgotten. In the second part, the cell tries to learn new information from the input to this cell. At last, in the third part, the cell passes the updated information from the current timestamp to the next timestamp.

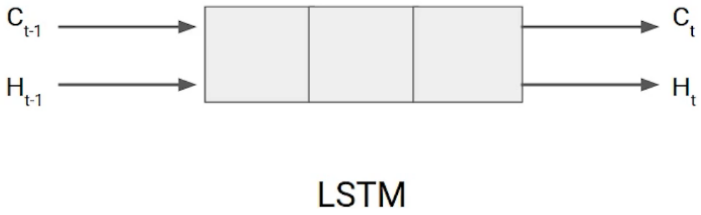
These three parts of an LSTM cell are known as gates. The first part is called Forget gate, the second part is known as the Input gate and the last one is the Output gate.

Just like a simple RNN, an LSTM also has a hidden state where H(t-1) represents the hidden state of the previous timestamp and Ht is the hidden state of the current timestamp. In addition to that LSTM also have a cell state represented by C(t-1) and C(t) for previous and current timestamp respectively.

Here the hidden state is known as Short term memory and the cell state is known as Long term memory. Refer to the following image.



Note that the cell state carries the information along with all the timestamps.



Bob is a nice person. Dan on the other hand is evil.

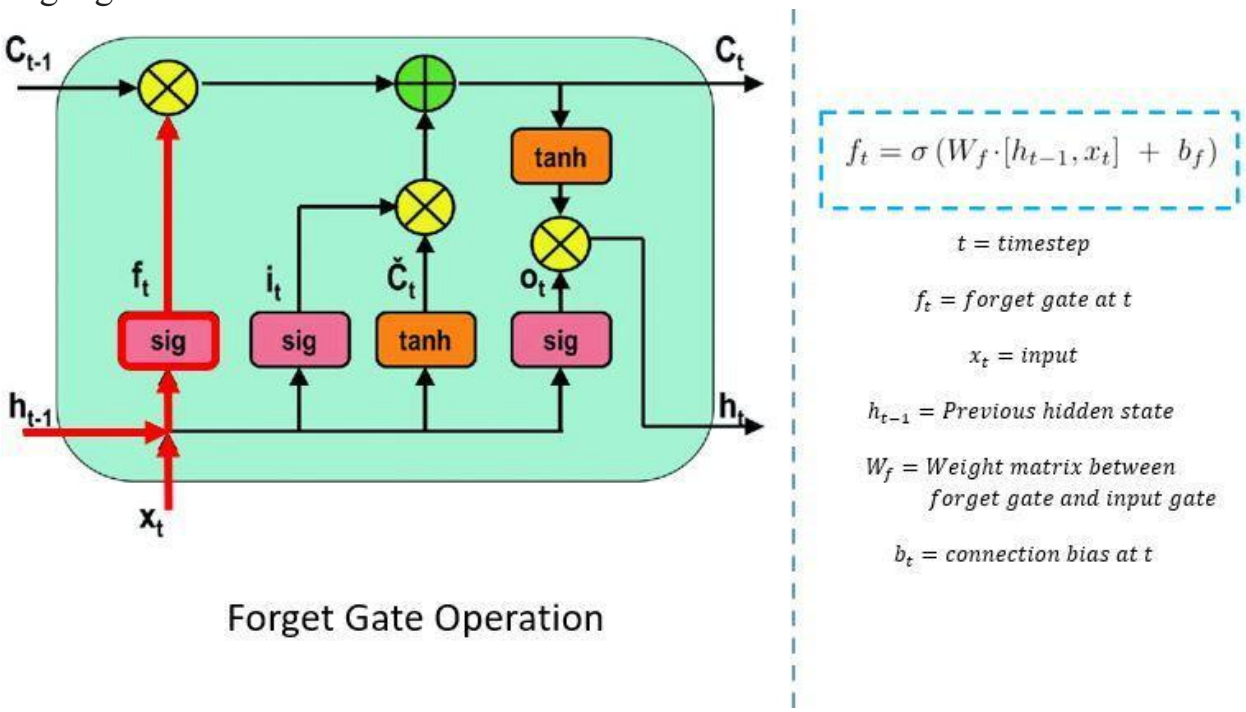
Let's take an example to understand how LSTM works. Here we have two sentences separated by a full stop. The first sentence is "Bob is a nice person" and the second sentence is "Dan, on the Other hand, is evil". It is very clear, in the first sentence we are talking about Bob and as soon as we encounter the full stop(.) we started talking about Dan.

As we move from the first sentence to the second sentence, our network should realize that we are no more talking about Bob. Now our subject is Dan. Here, the Forget gate of the network allows it to forget about it. Let's understand the roles played by these gates in LSTM architecture.

Eg. Naveen eats samosa almost every day. It shouldn't be hard to guess that his favorite cuisine is …..

Forget Gate

In a cell of the LSTM network, the first step is to decide whether we should keep the information from the previous timestamp or forget it. Here is the equation for forget gate.



Forget Gate Operation

The forget gate decides which information needs attention and which can be ignored. The information from the current input X(t) and hidden state h(t-1) are passed through the sigmoid function. Sigmoid generates values between 0 and 1. It concludes whether the part of the old output is necessary (by giving the output closer to 1). This value of f(t) will later be used by the cell for point-by-point multiplication.

$C_{t-1} * f_t = 0$ ...if $f_t = 0$ (forget everything)

$C_{t-1} * f_t = C_{t-1}$ ...if $f_t = 1$ (forget nothing)

If ft is 0 then the network will forget everything and if the value of ft is 1 it will forget nothing. Let's get back to our example, The first sentence was talking about Bob and after a full stop, the network will encounter Dan, in an ideal case the network should forget about Bob.

Input Gate

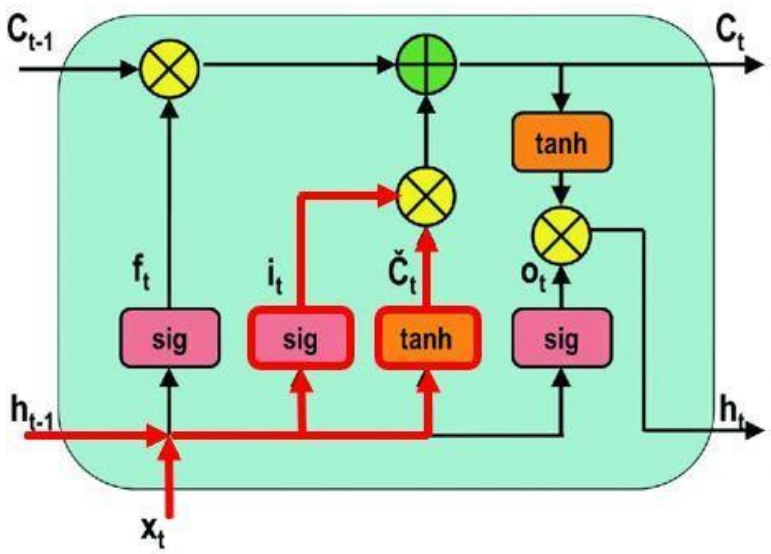
Let's take another example

"Bob knows swimming. He told me over the phone that he had served the navy for four long years."

So, in both these sentences, we are talking about Bob. However, both give different kinds of information about Bob. In the first sentence, we get the information that he knows swimming. Whereas the second sentence tells he uses the phone and served in the navy for four years.

Now just think about it, based on the context given in the first sentence, which information of the second sentence is critical. First, he used the phone to tell or he served in the navy. In this context, it doesn't matter whether he used the phone or any other medium of communication to pass on the information. The fact that he was in the navy is important information and this is something we want our model to remember. This is the task of the Input gate.

Input gate is used to quantify the importance of the new information carried by the input. Here is the equation of the input gate



Input Gate Operation

$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] + b_i\right)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$t = timestep$

$i_t = input\ gate\ at\ t$

$W_i = Weight\ matrix\ of\ sigmoid\ operator\ between\ input\ gate\ and\ output\ gate$

$b_t = bias\ vector\ at\ t$

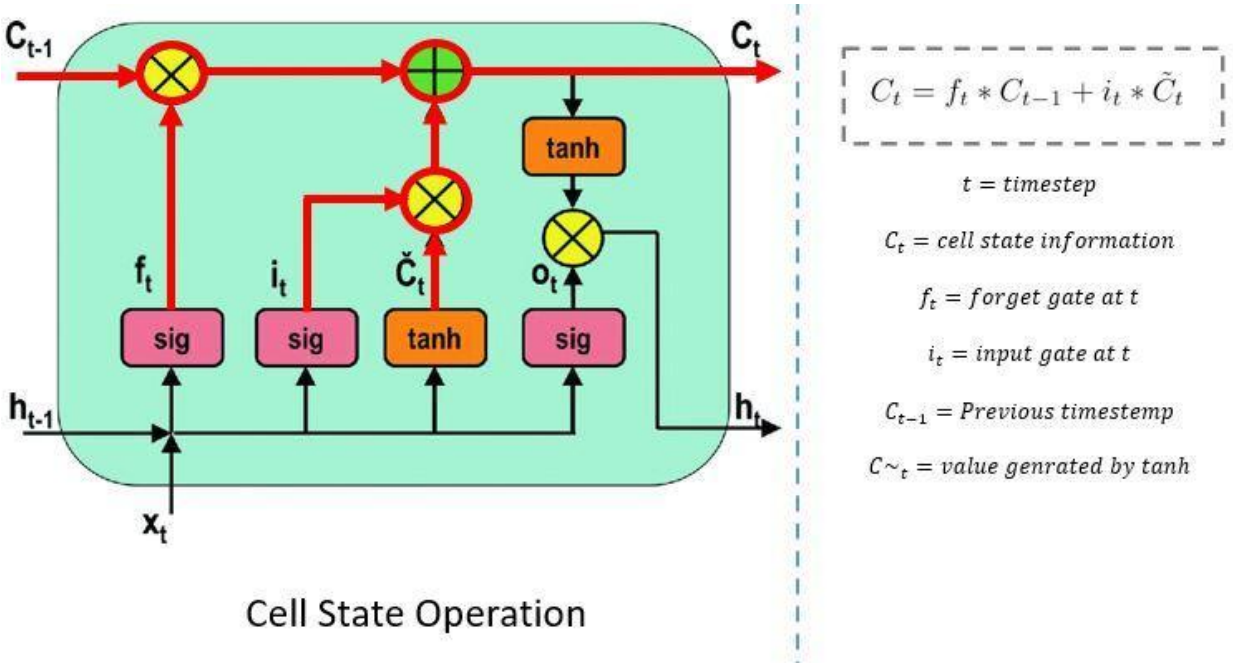
$C\sim_t = value\ genrated\ by\ tanh$

$W_c = Weight\ matrix\ of\ tanh\ operator\ between\ cell\ state\ information\ and\ network\ output$

$b_c = bias\ vector\ at\ t, w.r.t\ W_c$

The input gate performs the following operations to update the cell status. First, the current state X(t) and previously hidden state h(t-1) are passed into the second sigmoid function. The values are transformed between 0 (important) and 1 (not-important).
Next, the same information of the hidden state and current state will be passed through the tanh function. To regulate the network, the tanh operator will create a vector (C~(t)) with all the possible values between -1 and 1. The output values generated form the activation functions are ready for point-by-point multiplication.

Now the new information that needed to be passed to the cell state is a function of a hidden state at the previous timestamp t-1 and input x at timestamp t. The activation function here is tanh. Due to the tanh function, the value of new information will between -1 and 1. If the value is is negative the information is subtracted from the cell state and if the value is positive the information is added to the cell state at the current timestamp.



Cell State Operation

The network has enough information from the forget gate and input gate. The next step is to decide and store the information from the new state in the cell state. The previous cell state C(t-1) gets multiplied with forget vector f(t). If the outcome is 0, then values will get dropped in the cell state. Next, the network takes the output value of the input vector i(t) and performs point-by-point addition, which updates the cell state giving the network a *new cell state C(t).*

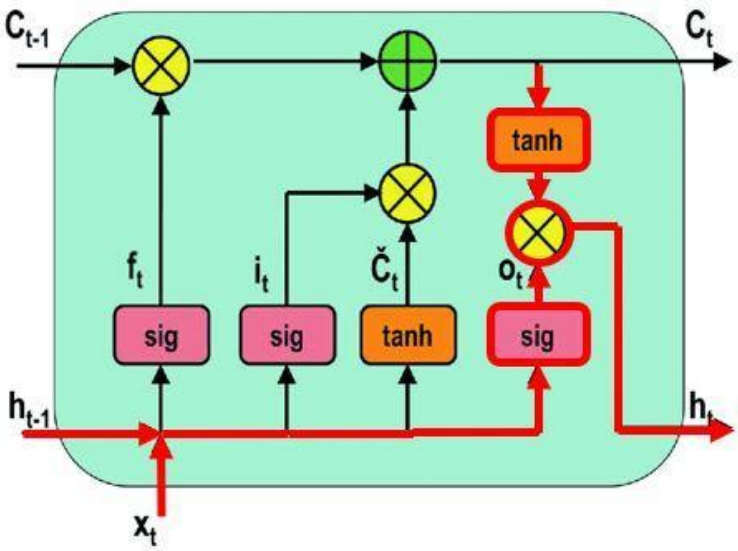
Output Gate

Now consider this sentence

"Bob single-handedly fought the enemy and died for his country. For his contributions, brave_____ ."

During this task, we have to complete the second sentence. Now, the minute we see the word brave, we know that we are talking about a person. In the sentence only Bob is brave, we cannot say the enemy is brave or the country is brave. So based on the current expectation we have to give a relevant word to fill in the blank. That word is our output and this is the function of our Output gate.

Here is the equation of the Output gate, which is pretty similar to the two previous gates.



Output Gate Operation

$$o_t = \sigma\left(W_o\ [h_{t-1}, x_t]\ +\ b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

$t = timestep$

$O_t = output\ gate\ at\ t$

$W_o = Weight\ matrix\ of\ output\ gate$

$b_o = bias\ vector, w.r.t\ W_o$

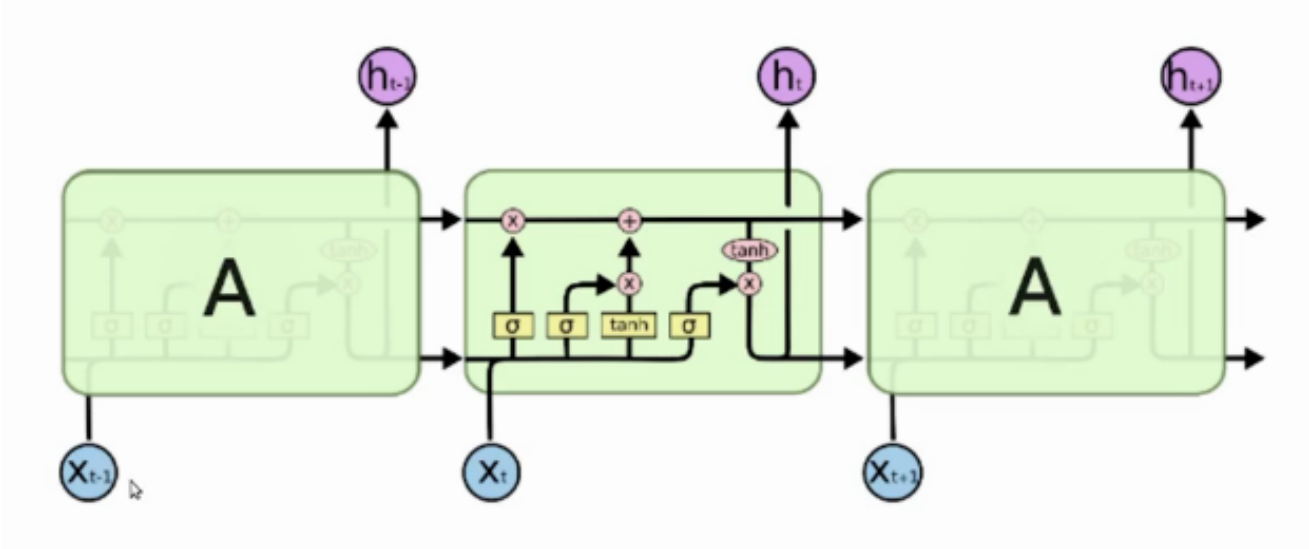
$h_t = LSTM\ output$

The output gate determines the value of the next hidden state. This state contains information on previous inputs.

First, the values of the current state and previous hidden state are passed into the third sigmoid function. Then the new cell state generated from the cell state is passed through the tanh function. Both these outputs are multiplied point-by-point. Based upon the final value, the network decides which information the hidden state should carry. This hidden state is used for prediction.

Finally, the new cell state and new hidden state are carried over to the next time step.

To conclude, the forget gate determines which relevant information from the prior steps is needed. The input gate decides what relevant information can be added from the current step, and the output gates finalize the next hidden state.



The vanishing gradient problem with LSTMs

Two main questions arise in Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks

1. Why do RNNs suffer from vanishing and exploding gradients?

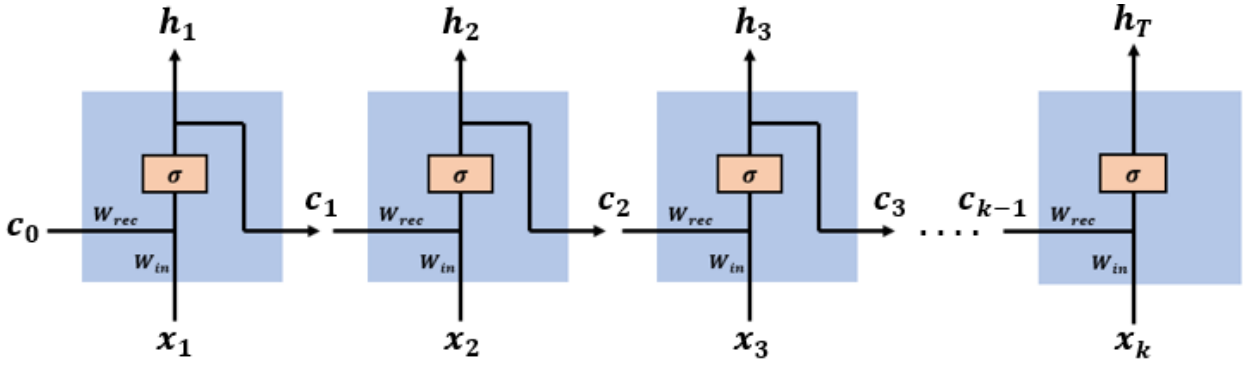
2. How do LSTMs keep the gradients from vanishing or explode?

RNNs and vanishing gradients

RNNs enable modelling time-dependent and sequential data tasks, such as stock market prediction, machine translation, text generation and many more.

However, RNNs suffer from the problem of vanishing gradients, which hampers learning of long data sequences. The gradients carry

information used in the RNN parameter update and when the gradient becomes smaller and smaller, the parameter updates become insignificant which means no real learning is done.



The network has an input sequence of vectors [x(1), x(2),..., x(k)], at time step t the network has an input vector x(t). Past information and learned knowledge is encoded in the network state vectors [c(1), c(2),..., c(k-1)], at time step t the network has an input state vector c(t-1). The input vector x(t) and the state vector c(t-1) are concatenated to comprise the complete input vector at time step t.

Backpropagation through time (BPTT) in RNNs

After the RNN outputs the prediction vector h(k), we compute the prediction error E(k) and use the Back Propagation Through time algorithm to compute the gradient

$$\frac{\partial E}{\partial W} = \sum_{t=1}^{T} \frac{\partial E_t}{\partial W}$$

The gradient is used to update the model parameters by:

$$W \leftarrow W - \alpha \frac{\partial E}{\partial W}$$

we have learning task that includes T time steps, the gradient of the error on the k time step is given by:

$$\frac{\partial E_k}{\partial W} = \frac{\partial E_k}{\partial h_k}\frac{\partial h_k}{\partial c_k}\cdots\frac{\partial c_2}{\partial c_1}\frac{\partial c_1}{\partial W}$$

for some time step k:

$$\frac{\partial E_k}{\partial W} \to 0$$

And our complete error gradient will vanish

$$\frac{\partial E}{\partial W} = \sum_{t=1}^{T}\frac{\partial E_t}{\partial W} \to 0$$

the network's weights update will be:

$$W \leftarrow W - \alpha\frac{\partial E}{\partial W} \approx W$$

So if we want not to vanish, our network needs to increase the likelihood that at least some of these sub gradients will not vanish, in other words, make the series of sub gradients not converge to zero.

In LSTN **the gradient contains the forget gate's vector of activations, which allows the network to better control the gradients values, at each time step, using suitable parameter updates of the forget gate.** The presence of the forget gate's activations allows the LSTM to decide, at each time step, that certain information should not be forgotten and to update the model's parameters accordingly.

for some time step k<T, we have that:

$$\sum_{t=1}^{k} \frac{\partial E_t}{\partial W} \rightarrow 0$$

Then for the gradient not to vanish, we can find a suitable parameter update of the forget gate at time step k+1 such that:

$$\frac{\partial E_{k+1}}{\partial W} \not\rightarrow 0$$

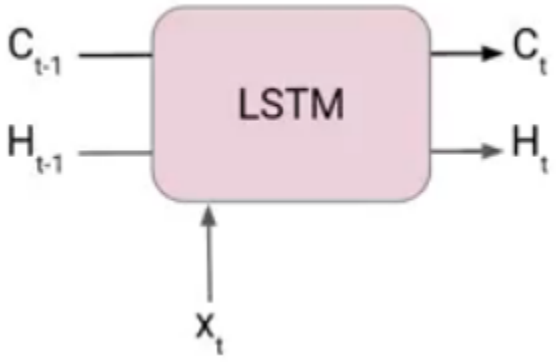
It is the presence of the forget gate's vector of activations in the gradient term along with additive structure which allows the LSTM to find such a parameter update at any time step, and this yields:

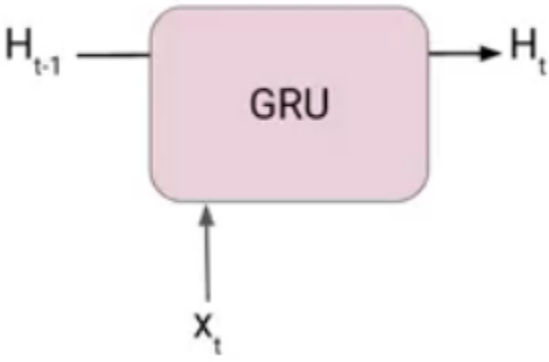
$$\sum_{t=1}^{k+1} \frac{\partial E_t}{\partial W} \not\rightarrow 0$$

and the gradient doesn't vanish.

Gated Recurrent Unit (GRU)

GRUs are very similar to Long Short Term Memory (LSTM). Just like LSTM, GRU uses gates to control the flow of information. They are relatively new as compared to LSTM. This is the reason they offer some improvement over LSTM and have simpler architecture.

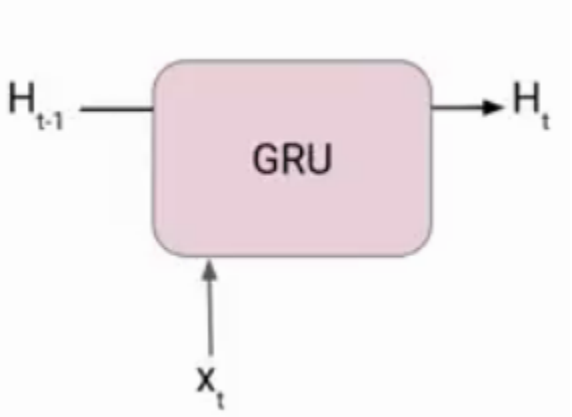




Unlike LSTM, it does not have a separate cell state (Ct). It only has a hidden state (Ht). Due to the simpler architecture, GRUs is faster to train.

The architecture of Gated Recurrent Unit

Here we have a GRU cell which more or less similar to an LSTM cell or RNN cell.



At each timestamp t, it takes an input X_t and the hidden state H_{t-1} from the previous timestamp t-1. Later it outputs a new hidden state H_t which again passed to the next timestamp.

Now there are primarily two gates in a GRU as opposed to three gates in an LSTM cell. The first gate is the Reset gate and the other one is the update gate.

Reset Gate (Short term memory)

The Reset Gate is responsible for the short-term memory of the network i.e the hidden state (Ht). Here is the equation of the Reset gate.

$$r_t = \sigma (x_t * U_r + H_{t-1} * W_r)$$

If you remember from the LSTM gate equation it is very similar to that. The value of $\mathbf{r_t}$ will range from 0 to 1 because of the sigmoid function. Here Ur and Wr are weight matrices for the reset gate.

Update Gate (Long Term memory)

Similarly, we have an Update gate for long-term memory and the equation of the gate is shown below.

$$u_t = \sigma (x_t * U_u + H_{t-1} * W_u)$$

The only difference is of weight metrics i.e Uu and Wu.

How GRU Works

Now let's see the functioning of these gates. To find the Hidden state H_t in GRU, it follows a two-step process. The first step is to generate what is known as the candidate hidden state. As shown below

Candidate Hidden State

$$\hat{H}_t = \tanh(x_t * U_g + (r_t \circ H_{t-1}) * W_g)$$

It takes in the input and the hidden state from the previous timestamp t-1 which is multiplied by the reset gate output r_t. Later passed this entire information to the tanh function, the resultant value is the candidate's hidden state.

The most important part of this equation is how we are using the value of the reset gate to control how much influence the previous hidden state can have on the candidate state.

If the value of r_t is equal to 1 then it means the entire information from the previous hidden state H_{t-1} is being considered. Likewise, if the value of r_t is 0 then that means the information from the previous hidden state is completely ignored.

Hidden state

Once we have the candidate state, it is used to generate the current hidden state H_t. It is where the Update gate comes into the picture. Now, instead of using a separate gate like in LSTM, in GRU we use a single update gate to control both the historical information which is H_{t-1} as well as the new information which comes from the candidate state.

$$H_t = u_t \circ H_{t-1} + (1-u_t) \circ \hat{H}_t$$

Now assume the value of u_t is around 0 then the first term in the equation will vanish which means the new hidden state will not have much information from the previous hidden state. On the other hand, the second part becomes almost one that essentially means the hidden state at the current timestamp will consist of the information from the candidate state only.

$$H_t = u_t \circ H_{t-1} + (1-u_t) \circ \hat{H}_t$$

Similarly, if the value of u_t is 1, the second term will become entirely 0 and the current hidden state will entirely depend on the first term i.e the information from the hidden state at the previous timestamp t-1.

$$H_t = u_t \circ H_{t-1} + (1-u_t) \circ \hat{H}_t$$

Hence we can conclude that the value of u_t is very critical in this equation and it can range from 0 to 1.